Abdulmohsen Almunayes

RED ID: 823343580

3/14/2024

LAB 5 REPORT

What does your program do?

My code produces musical tones based on the buttons pressed on the keypad, adjusting the period to achieve a 50% duty cycle.

What variables/registers does your program use?

My program uses registers TCCR0A/B, OCR0A/B, TIFR0, DDRB/D, OCF0A/B PORTB, PINB, and PORTD. The program also uses 2D integer array called notes. The program also has a Check keypad function and a Play Tone function.

What is the main algorithm/logic of your program?

Initially, the code configures the data direction and sets the bits for the keypad on PORTD and PORTB to 1 and it also sets PD1 pin as output for the AUX. Then, within an infinite loop, it invokes the checkKeypad function. Upon detecting a button press, the program identifies the pressed button, adjusts OCR0A accordingly, and sets OCR0B to half of OCR0A. Subsequently, a musical note is generated through a PWM setup, where OCR0B determines the duration of sound emission per cycle, and OCR0A defines the cycle's period. This approach allows for the production of various sound frequencies.

What external hardware connections did you make?

Connections were made between the PC and the microcontroller, from the microcontroller to the keypad, and lastly from the microcontroller to the AUX port.

C CODE

```
#define F_CPU 16000000UL

#include <util/delay.h>
#include <avr/io.h>

int notes[4][4] = {
    {141, 131, 126, 119},
    {112, 105, 99, 94},
    {89, 83, 79, 74},
    {70, 66, 62, 59}
};

void playTone(int index) {
    while (!(PINB & (1 << index))) {
        while (!(TIFR0 & (1 << OCF0B))); // Wait for overflow flag
        PORTD |= (1 << 1); // Set speaker output HIGH
        TIFR0 |= (1 << OCF0B); // Reset overflow flag
        while (!(TIFR0 & (1 << OCF0A))); // Wait for overflow flag
        PORTD &= ~(1 << 1); // Set speaker output LOW
        TIFR0 |= (1 << OCF0A); // Resets overflow flag
    }
}

void checkKeypad(void) {
    int index;
    int row;
    int column;

    for (row = 4; row < 8; row++) {
        PORTD &= ~(1 << row); // Set each row low
        for (column = 0; column < 4; column++) {
            if (!(PINB & (1 << column))) { // Check each column
                index = notes[row - 4][column];
                OCR0A = index;// Output compare register
                OCR0B = OCR0A / 2; // 50% initial duty cycle
                playTone(column);
            }
        }
        PORTD |= (1 << row); // Sets each row back to high after checking
    }
}
```

```
int main(void) {

   DDRD |= (1 << DDD4 | 1 << DDD5 | 1 << DDD6 | 1 << DDD7); // Sets bits 4, 5, 6, and 7 of
the DDRD register to 1, configuring these pins as outputs.
   DDRB &= ~(1 << DDB0 & 1 << DDB1 & 1 << DDB2 & 1 << DDB3); // Clears bits 0, 1, 2,
and 3 of the DDRB register, configuring these pins as inputs.
   PORTD |= (1 << 4 | 1 << 5 | 1 << 6 | 1 << 7); // Sets bits 4, 5, 6, and 7 of the PORTD register
to 1, enabling pull-up resistors for pins D4-D7.
   PORTB |= (1 << 0 | 1 << 1 | 1 << 2 | 1 << 3); // Sets bits 0, 1, 2, and 3 of the PORTB register
to 1, enabling pull-up resistors for pins B0-B3.
   TCCR0A |= (1 << WGM01); // Set to CTC mode
   TCCR0B |= (1 << CS02); // Set prescale to 256
   DDRD |= (1 << 1);

   while (1) {
      checkKeypad();
   }
}
```